# Architecting a Distributed Data Platform: A Comprehensive Blueprint for Next.js and Go Fiber Integration

The contemporary landscape of enterprise software engineering demands system architectures that are exceptionally resilient, highly scalable, and fundamentally decoupled. The architecture under analysis presents a sophisticated paradigm for a centralized data platform, engineered to aggregate, transform, and expose data from disparate sources, including MySQL, PostgreSQL, and dynamic external REST APIs. To achieve the requisite performance and developer velocity, the system integrates a modern frontend stack—utilizing Next.js 15, the Bun runtime, and Shadcn UI—with a high-throughput backend constructed in Go, leveraging the Fiber v3 framework.

Crucially, the architecture delegates identity and access management to the Better Auth framework, establishing a secure, framework-agnostic session boundary. Data persistence and external integrations are orchestrated via robust Object-Relational Mapping (ORM) using GORM, combined with a highly adaptable REST API client pattern designed specifically for dynamic authentication header injection. The deployment and local orchestration of this distributed stack are governed by Docker Compose, with planned enhancements including Redis-backed memory caching and default data encryption.

This exhaustive research report deconstructs this architectural blueprint, providing deep analytical insights into the implementation mechanics, performance trade-offs, and design patterns required to manifest this system in a production environment.

## Frontend Ecosystem: Next.js 15, Bun Runtime, and Shadcn UI

The presentation layer of this data platform is designed to provide a highly interactive, server-side rendered (SSR) dashboard interface. To achieve optimal performance and maintainability, the architecture specifies the convergence of Next.js, the Bun runtime, and the Shadcn UI component registry.

### The Role of the Bun Runtime

Historically, Node.js and its accompanying package managers (npm, yarn) have dominated the JavaScript ecosystem. However, the adoption of Bun introduces a paradigm shift in execution speed and dependency resolution.[1] Bun acts as a drop-in replacement for Node.js, encompassing a JavaScript runtime, a bundler, a transpiler, and a package manager within a

single executable.

When initializing the Next.js project, the execution of commands via Bun (e.g., bunx --bun shadcn@latest init -t next) bypasses the traditional Node.js execution layer, significantly accelerating the scaffolding process.[2] Furthermore, in complex monorepo environments or projects with intricate dependency trees, Bun resolves peer dependency conflicts—such as the ERESOLVE errors frequently encountered when mixing modern React 19 libraries with legacy packages—with far greater efficiency than traditional package managers.[1]

### Architectural Philosophy of Shadcn UI

Unlike traditional component libraries (e.g., Material UI, Chakra UI) that are installed as opaque npm dependencies, Shadcn UI operates as a component registry.[2] It embraces an ownership model where the source code for highly accessible, customizable components is copied directly into the application's repository, typically under a src/components/ui directory.[1]

This approach yields profound benefits for an enterprise data platform:

1. **Zero Vendor Lock-in:** Because the raw React and Tailwind CSS v4 code resides within the application, developers possess absolute authority to modify the underlying markup and logic.[1]
2. **Granular Opt-in:** The bunx --bun shadcn@latest add button command ensures that the application only imports the specific components it requires, eliminating the bundle bloat associated with monolithic UI frameworks.[2]
3. **Accessibility Foundation:** Under the hood, Shadcn UI relies on Radix UI primitives. These unstyled primitives handle complex accessibility requirements—such as keyboard navigation, focus management, and ARIA attribute toggling—ensuring the dashboard is compliant with enterprise accessibility standards.[1]

The integration of thematic toggling (dark/light mode) is achieved seamlessly through the next-themes package, which wraps the application layout in a context provider, preventing hydration mismatches when server-rendered HTML encounters client-side theme preferences.[1]

## Unified Identity and Access Management: Better Auth Implementation

A critical requirement for any data management platform is a rigorous authentication and authorization boundary. The architecture designates Better Auth for this purpose. Better Auth is a modern, comprehensive, and framework-agnostic authentication library built fundamentally for the TypeScript ecosystem.[4]

### Core Mechanics and Configuration

Better Auth distinguishes itself by offering a vast array of functionalities out-of-the-box, negating the need to stitch together disparate libraries for credentials, OAuth2, and session management. It supports traditional email/password login, social sign-on (Google, GitHub), Two-Factor Authentication (2FA), and complex organizational multi-tenancy (RBAC) via its plugin ecosystem.[6]

The instantiation of the auth server typically resides in a configuration file (e.g., lib/auth.ts), where the database adapter is defined. Better Auth seamlessly integrates with ORMs like Prisma or Drizzle to manage its core tables: users, sessions, and accounts.[5]

| **Configuration Parameter** | **Data Type** | **Default Behavior** | **Strategic Implication** |
|---|---|---|---|
| emailAndPassword.enabled | Boolean | false | Must be explicitly enabled to allow traditional credential-based sign-up flows.[9] |
| session.expiresIn | Integer | 7 days | Defines the absolute maximum lifespan of an authenticated session before requiring re-authentication.[10] |
| session.updateAge | Integer | 1 day | Establishes a threshold; if a session is utilized after this duration, its expiration is automatically extended (sliding window).[10] |
| advanced.cookieCache | Object | Disabled | When enabled, caches session data directly within an encrypted cookie payload, |

| | | | |
|---|---|---|---|
| | | | facilitating stateless or reduced-I/O architectures.[11] |
| plugins | Array | `` | Allows for the injection of specialized modules, such as nextCookies() for automated cookie header management in Next.js App Router.[9] |

## Backend Session Validation (The Next.js to Go Bridge)

A significant architectural challenge arises from the dichotomy of the stack: the frontend and initial authentication handler reside in the Node.js/Bun ecosystem (Next.js), while the core data processing API resides in Go. The Go Fiber backend must cryptographically verify that incoming HTTP requests originated from an authenticated user.

Because Better Auth relies on standard HTTP-only cookies, the architecture must ensure these cookies are forwarded from the Next.js client to the Go backend.[14] Once the Go backend receives the request, it must validate the session. The architecture supports two primary methodologies for this validation:

1. **Stateful Database Validation (The Default):** The Go backend extracts the session_token from the incoming cookie or Authorization header. It then executes a high-speed database query against the shared PostgreSQL database (specifically the Better Auth session table) to confirm the token's validity, expiration, and associated user ID.[6] While this guarantees immediate session revocation (if an admin bans a user, the next database check fails), it introduces database I/O latency on every protected API route.
2. **Stateless JWT Validation (The Optimization):** To circumvent the database bottleneck, Better Auth can be configured to operate in stateless mode via the JWT plugin.[12] In this paradigm, Better Auth signs a JSON Web Token containing the user's identity and permissions. The Go backend, possessing the shared symmetric or asymmetric cryptographic key, parses and verifies the JWT's signature and expiration entirely in-memory.[15] This drastically increases the throughput of the Go backend, though it introduces the inherent complexity of JWT revocation delays.

For maximum performance, the stateless JWT approach, combined with a Redis-backed blocklist for revoked tokens, represents the optimal configuration for this high-traffic data platform.[16]

# Backend Foundation: Go Fiber v3 and Clean Architecture

The engine of the data platform is constructed using Go v1.25+ and the Fiber v3 framework. Fiber is an Express-inspired web framework built atop fasthttp, a high-performance HTTP engine for Go.[17] The v3 iteration of Fiber introduces profound architectural enhancements that directly address the complexities of building enterprise APIs.

## Fiber v3 Innovations

The migration to Fiber v3 necessitates the adoption of modern Go paradigms. One of the most significant upgrades is the native implementation of Go's context.Context interface within the fiber.Ctx object.[18] In previous iterations, developers were forced to manually bridge contexts (e.g., c.UserContext()) to pass request-scoped cancellation signals down to database drivers. In v3, passing the Fiber context directly into a database query ensures that if a client prematurely disconnects or a timeout is reached, the underlying database query is immediately cancelled, preventing resource exhaustion.[18]

Furthermore, Fiber v3 revolutionizes middleware development through its extractors package. Previously, custom logic was required to hunt for authentication tokens across headers, cookies, and query parameters. The extractors.FromAuthHeader() utility standardizes this, providing a unified, secure, and highly performant mechanism for token retrieval across the entire routing tree.[20]

## Implementing Clean Architecture

To prevent the backend from degrading into a tightly coupled monolith, the architecture strictly adheres to Clean Architecture principles.[22] This architectural style enforces the separation of concerns, dividing the application into concentric layers where dependencies invariably point inward toward the domain logic.

1. **Delivery Layer (Handlers):** Housed within the internal/handler/ directory, this layer is solely responsible for interacting with the Fiber framework.[23] It parses incoming HTTP requests, decodes JSON payloads, invokes the underlying business logic, and formats the HTTP responses. It contains no business rules.
2. **Domain / Usecase Layer:** Residing in internal/usecase/, this is the heart of the application. It dictates the business rules—such as how data from multiple databases should be merged or transformed.[24] This layer operates entirely independent of the HTTP framework or the specific database technology.
3. **Data Access / Repository Layer:** Located in internal/repository/, this layer encapsulates all interactions with persistent storage (PostgreSQL via GORM) or external APIs (via REST clients).[22]

By defining interfaces in the Domain layer that the Repository layer implements, the system achieves true orthogonality. If the data platform decides to migrate from PostgreSQL to a distributed SQL database like CockroachDB, only the specific repository implementation requires modification; the business logic and HTTP handlers remain entirely untouched.[24]

# Database Mastery: GORM and Intelligent Connection Pooling

The architectural diagram explicitly highlights a critical operational challenge: *"how to make a smart db open connection and automatic close"*. Addressing this requires a deep understanding of how Go and GORM manage database connections at the kernel level.

## The Fallacy of Manual Connection Management

A common misconception among developers transitioning to Go is the belief that a database connection should be manually opened at the start of an HTTP request and explicitly closed at the end using defer db.Close().[26] In a high-throughput backend, this approach is disastrous. Establishing a TCP connection to PostgreSQL, completing the TLS handshake, and authenticating requires significant latency (often 20-100ms).[28] Doing this per-request will swiftly exhaust the database server's CPU and memory resources, leading to connection storms.[29]

## The Mechanics of "Smart" Automatic Pooling

The solution relies on the inherent pooling capabilities of Go's standard database/sql package, which GORM utilizes seamlessly under the hood.[30] A *gorm.DB instance does not represent a single database connection; it represents a thread-safe connection pool.[31]

The application must instantiate the GORM database connection *exactly once* during application startup.[32] From that point forward, the "smart open and automatic close" behavior is governed entirely by configuring the connection pool parameters:

| Pool Setting | Go Standard Library Method | Architectural Function and Strategy |
|---|---|---|
| **Max Open Connections** | db.DB().SetMaxOpenConns (n) | Imposes a hard limit on the total number of simultaneous connections to the database. If traffic spikes exceed this limit, Go |

| | | |
|---|---|---|
| | | automatically places incoming queries into a queue, forcing them to wait until a connection is freed. This acts as a critical backpressure mechanism, preventing the application from crashing the PostgreSQL server.[33] |
| **Max Idle Connections** | db.DB().SetMaxIdleConns(n) | Dictates how many connections are kept alive in the pool after a query finishes. A higher number allows the application to handle sudden bursts of traffic instantly, as connections are already established and waiting in memory.[33] |
| **Connection Max Lifetime** | db.DB().SetConnMaxLifetime(d) | The true mechanism for "automatic closing." This setting dictates the absolute maximum duration a connection can exist before the pool forcibly closes it and opens a fresh one. This is vital for navigating load balancers, proxies, and firewalls that quietly terminate long-lived idle TCP connections, preventing opaque connection errors.[30] |

When a repository method calls db.Find(&users), GORM requests a connection from the pool. Once the data is retrieved and the rows are closed, the connection is automatically returned to the idle pool—no manual Close() required.[26]

## Dynamic Database Routing and Multi-Tenancy

The architecture specifies the use case: *"Manage central data (ex: an org has many db)"*.

Handling multiple distinct database connections dynamically introduces severe complexity. Attempting to maintain a separate connection pool for hundreds of tenant databases will instantly exhaust the host's file descriptors and RAM.[36]

To solve this efficiently, the architecture leverages GORM's dynamic capabilities, specifically utilizing transaction scopes. Rather than switching physical connection pools, the application utilizes a single, massive connection pool connected to the primary PostgreSQL cluster. When a request arrives, middleware identifies the tenant. The repository layer then utilizes a GORM scope to dynamically prefix the table name or alter the PostgreSQL search path (e.g., SET search_path TO tenant_a) just prior to executing the query.[37]

This allows GORM to dynamically route queries to the correct logical isolation layer while reusing the same highly optimized, underlying TCP connection pool.[37]

# External Ingestion: The REST API Adapter Pattern with Dynamic Auth

One of the most complex requirements explicitly requested in the architecture is the setup of a REST API data source capable of configuring authentication headers dynamically. Enterprise data platforms rarely exist in isolation; they must ingest data from third-party systems, each employing disparate security mechanisms (e.g., rotating API keys, OAuth2 Bearer tokens, HMAC signatures).[39]

## The Structural Adapter Pattern

To achieve this without polluting the core business logic, the system utilizes the Adapter Design Pattern.[41] The Adapter acts as a translational bridge between the application's domain needs and the specific, idiosyncratic requirements of an external REST API.[42]

The architecture is composed of four distinct entities:

1. **The Client (Domain Usecase):** The core business logic that requires the data. It expects a clean, standardized interface.
2. **The Target Interface:** A Go interface defined in the domain layer (e.g., FetchAnalyticsData(ctx context.Context) (Data, error)).
3. **The Adapter:** A concrete struct in the repository layer that implements the Target Interface.
4. **The Adaptee:** The external REST API and the HTTP client used to invoke it.[41]

## Dynamic Header Injection via Fiber v3 Client

In older Go applications, developers often instantiated a new http.Client and http.Request manually for every call, leading to extreme code duplication and inconsistent timeout policies.[43]

Fiber v3 resolves this by introducing a highly ergonomic client package.

The system initializes a long-lived Client Factory upon startup. This base client contains static configurations that apply to all requests sent to that specific provider, such as the BaseURL, TLS configurations, and global timeouts.[44]

However, because authentication headers (like short-lived OAuth tokens) are dynamic and request-specific, they cannot be baked into the base client. Instead, the Adapter utilizes Fiber's client.Config struct, which permits per-request overrides.[45]

The implementation logic follows a precise sequence:

1. **Authentication Resolution:** The Adapter utilizes an injected AuthenticationProvider interface to resolve the current, valid token. This provider handles the complexities of checking cache, executing OAuth token refreshes if necessary, and returning the raw string.[46]
2. **Request Execution:** The Adapter calls the specific endpoint using the Fiber client, passing the resolved token into the client.Config header map.

Go

```go
// 1. Base Client instantiated once
var salesForceClient = client.New().
  SetBaseURL("https://api.salesforce.com/v2").
  SetTimeout(10 * time.Second)

// 2. The Adapter Implementation
type SalesforceAdapter struct {
  client   *client.Client
  auth     AuthenticationProvider // Interface handling token refresh/cache
}

// 3. Implementing the Target Interface
func (a *SalesforceAdapter) FetchAnalyticsData(ctx context.Context, tenantID string) (Data, error) {
  // Dynamically retrieve the specific auth header required for this request/tenant
  authHeader, err := a.auth.GetAuthorizationHeader(ctx, tenantID)
  if err!= nil {
    return nil, fmt.Errorf("auth resolution failed: %w", err)
  }

  var result ResponseDTO
```

```go
    // Execute the request, dynamically injecting the header via client.Config
    resp, err := a.client.Get("/analytics", client.Config{
        Header: map[string]string{
            "Authorization": authHeader,      // Dynamic Injection
            "Accept":        "application/json",
        },
    })

    // Response parsing and error handling...
    return result.Data, nil
}
```

This structural pattern ensures maximum orthogonality. If the external REST API mandates a shift from basic API keys to OAuth2, the only component requiring modification is the isolated AuthenticationProvider. The base client, the Adapter's mapping logic, and the core domain UseCase remain completely unaffected.[46]

# Taming the Data: Dynamic JSON and Streaming Heavy Payloads

As a centralized data platform, the application inevitably interfaces with dynamic data structures and heavily bloated payloads. The architecture must anticipate scenarios where the exact schema of an incoming JSON payload is unknown at compile time, and situations where the payload size threatens the memory limits of the container.

## Navigating Unknown Schemas: The Cost of Reflection

Go is a statically typed language, which inherently conflicts with the dynamic, unpredictable nature of arbitrary JSON. The architecture explicitly notes the issue: *"How to handle data mapping since we use dynamic data"*.[47]

The standard approach to decoding unknown JSON is to unmarshal the payload into a map[string]interface{} (or map[string]any).[47] This allows the application to ingest any structure, navigating the resulting nested maps and slices via type assertions (e.g., val, ok := data["key"].(string)).[47]

However, this flexibility comes at a severe performance cost. The internal mechanics of Go's encoding/json rely extensively on the reflect package.[49] Reflection allows the Go runtime to inspect, decode, and manipulate types dynamically during execution.[50] Profiling high-throughput Go applications frequently reveals that reflection is one of the most expensive operations available. It circumvents compile-time optimizations, resulting in dynamic lookups,

excessive pointer chasing, and heavy allocations on the heap.[51] When thousands of operations per second hit reflection-heavy code, Garbage Collection (GC) pressure spikes, severely degrading the P99 latency of the API.[51]

To optimize mapping performance for dynamic data, the architecture employs strategic alternatives:

1. **Partial Static Structs with json.RawMessage:** When parts of the JSON schema are known but others are highly dynamic, the system utilizes json.RawMessage. This instructs the JSON decoder to capture a specific JSON sub-tree as a raw byte slice (byte), bypassing the reflection engine entirely for that segment.[47] The raw bytes can then be stored directly in the database or processed lazily only when absolutely necessary.
2. **Code Generation:** For known schemas experiencing extreme load, the architecture avoids the standard encoding/json package entirely, utilizing code-generation tools like easyjson. This analyzes the structs at compile time and generates hardcoded, reflection-free parsing routines, yielding zero-allocation unmarshaling.[51]

## Streaming Massive Data Sets

The second major hurdle identified in the architecture is *"How to handle getting heavy data"*. If the REST API adapter requests a 500MB JSON payload from a legacy system, and attempts to load that payload entirely into RAM before returning it to the frontend, the Go service will likely be terminated by the operating system for Out-Of-Memory (OOM) violations.[53]

To safeguard memory integrity, the architecture abandons buffering in favor of data streaming. Fiber v3 introduces native buffered streaming support through the SendStreamWriter method attached to the context.[54]

Instead of accumulating the payload, the adapter establishes the HTTP connection to the external API and immediately returns an io.Reader representing the response body. The delivery layer then utilizes SendStreamWriter to pipe this data directly to the client in highly controlled, small byte chunks.[56]

Go

```go
app.Get("/export-heavy-data", func(c fiber.Ctx) error {
  // Assume adapter returns the active io.Reader of the external API response
  dataReader, err := repository.FetchHeavyDataStream(c)

  c.Set("Content-Type", "application/json")
  c.Set("Transfer-Encoding", "chunked")
```

```go
  return c.SendStreamWriter(func(w *bufio.Writer) {
    buffer := make(byte, 4096) // Process strictly in 4KB chunks
    for {
      n, err := dataReader.Read(buffer)
      if n > 0 {
        w.Write(buffer[:n])
        w.Flush() // Instantly transmit chunk to Next.js client
      }
      if err == io.EOF {
        break
      }
    }
  })
})
```

Because the data is processed in isolated 4KB increments and instantly flushed over the network via the *bufio.Writer, the memory consumption of the Go container remains entirely flat and predictable, regardless of whether it is transferring a 10MB or 10GB payload.[53]

# Infrastructure Orchestration and Future Enhancements

The deployment of this decoupled architecture requires precise orchestration to ensure environment parity and reliable startup sequences. The system relies on Docker Compose to declaratively manage the infrastructure footprint.[58]

## Docker Compose Topography

The docker-compose.yml file defines a highly isolated network topography:

1. **Next.js Frontend Container:** Built using a multi-stage Dockerfile to trim unneeded dependencies, compiled with Bun, and exposed exclusively to the external network via port 3000.[59]
2. **Go Fiber Backend Container:** Compiled as a static binary into a distroless or minimal Alpine image, representing an incredibly small attack surface. It exposes its API on internal port 8080.[17]
3. **PostgreSQL Database Container:** The primary persistent store, configured with persistent volume claims (/var/lib/postgresql/data) to prevent data loss upon container destruction.[61]
4. **Redis Container:** An in-memory data store utilized for caching operations and stateless Better Auth session management.[58]

Crucially, the Docker Compose file utilizes the depends_on directive combined with health checks. This guarantees that the Go Fiber backend will not attempt initialization until the PostgreSQL database has fully started and is ready to accept connections, eliminating race conditions and crash loops during deployment.[61]

### Future Architectural Iterations

As the platform matures, the architecture dictates several critical enhancements:

- **Comprehensive Caching Strategy:** While database pooling mitigates connection overhead, querying disk still incurs latency. The system plans to implement the Cache-Aside pattern utilizing the integrated Redis container. The Go backend will intercept high-frequency read queries, checking Redis first; upon a cache miss, the data will be fetched from PostgreSQL, serialized, and cached in Redis with a defined Time-To-Live (TTL).[63]
- **Default Encryption at Rest:** Given the sensitive nature of centralized data, the architecture proposes default encryption. This will be implemented at the application layer utilizing GORM plugins (e.g., gorm-crypto), which transparently encrypt specific struct fields (like PII) using AES-GCM before writing to the database, ensuring that even if the physical database volume is compromised, the data remains cryptographically secure.[64]

## Conclusion

The system architecture analyzed herein represents a formidable blueprint for a modern, high-throughput distributed data platform. By fusing the rapid developmental ergonomics of Next.js, Bun, and Shadcn UI with the extreme raw performance of a Go Fiber backend, the platform achieves an optimal equilibrium between developer velocity and production scalability.

The strategic integration of Better Auth yields a highly robust, framework-agnostic security perimeter capable of scaling to multi-tenant enterprise requirements. On the backend, the rigid adherence to Clean Architecture principles ensures the business logic remains entirely decoupled from external volatility.

By masterfully tuning database/sql connection pools and utilizing GORM transaction scopes, the system achieves dynamic database routing without resource exhaustion. Furthermore, the implementation of the Adapter pattern via Fiber's highly configurable client successfully tames the chaos of consuming disparate REST APIs with fluctuating authentication demands. Finally, by abandoning deep reflection in favor of raw JSON handling, and replacing memory buffering with buffered data streaming, the architecture guarantees stability and sub-millisecond latencies, culminating in an enterprise-grade data orchestration engine.

### Works cited

1. Building a Modern Application 2025: A Complete Guide for Next.js 15, React 19, Tailwind CSS v4, and Shadcn. - Medium, accessed on March 10, 2026, https://medium.com/@dilit/building-a-modern-application-2025-a-complete-guide-for-next-js-1b9f278df10c
2. Next.js - Shadcn UI, accessed on March 10, 2026, https://ui.shadcn.com/docs/installation/next
3. Setup Next.js 14, Shadcn UI, Prisma | by Aji Maulana | Medium, accessed on March 10, 2026, https://medium.com/@aji658911/setup-next-js-14-shadcn-ui-fa8f419f7f6e
4. Better Auth, accessed on March 10, 2026, https://better-auth.com/
5. Complete Better Auth Integration Guide for Next.js with Prisma - Authentication, Social Login & Password Reset – JB, accessed on March 10, 2026, https://jb.desishub.com/blog/nextjs-better-auth
6. Better Auth, accessed on March 10, 2026, https://www.better-auth.com/
7. Better Auth Full Tutorial with Next.js, Prisma ORM, PostgreSQL, Nodemailer - Reddit, accessed on March 10, 2026, https://www.reddit.com/r/nextjs/comments/1kllpsv/better_auth_full_tutorial_with_nextjs_prisma_orm/
8. Better Auth Starter – A Next.js 15 authentication boilerplate with BetterAuth (credentials & Google login), Tailwind CSS, ShadCN, Prisma ORM, and PostgreSQL. Secure & scalable auth solution! · GitHub, accessed on March 10, 2026, https://github.com/devAaus/better-auth
9. Better-Auth with Next.js — A Complete Guide for Modern Authentication | by Amit upadhyay, accessed on March 10, 2026, https://medium.com/@amitupadhyay878/better-auth-with-next-js-a-complete-guide-for-modern-authentication-06eec09d6a64
10. BetterAuth with Encore.ts - Complete Backend Guide, accessed on March 10, 2026, https://encore.dev/blog/betterauth-tutorial
11. Options | Better Auth, accessed on March 10, 2026, https://www.better-auth.com/docs/reference/options
12. Session Management | Better Auth, accessed on March 10, 2026, https://www.better-auth.com/docs/concepts/session-management
13. Next.js integration - Better Auth, accessed on March 10, 2026, https://better-auth.com/docs/integrations/next
14. Using better-auth on Nextjs with existing backend (not within Nextjs api) #5336 - GitHub, accessed on March 10, 2026, https://github.com/better-auth/better-auth/discussions/5336
15. Authentication and Authorization (OpenID Connect)of Go Rest Api's using an open-source IAM called Keycloak | by Allu Sai Prudhvi | Medium, accessed on March 10, 2026, https://medium.com/@allusaiprudhvi999/authentication-and-authorization-in-golang-microservice-using-an-open-source-iam-called-keycloak-46f03a26248f
16. Bridging the Gap: Implementing Better Auth with Non-Node Backends - DEV Community, accessed on March 10, 2026, https://dev.to/rogasper/bridging-the-gap-implementing-better-auth-with-non-no

de-backends-5dh4
17. How to Build Production-Ready REST APIs with Go Fiber and PostgreSQL - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-25-production-rest-apis-go-fiber-postgresql/view
18. What's New in Fiber v3, accessed on March 10, 2026, https://docs.gofiber.io/blog/whats-new-in-fiber-v3
19. How to manage database timeouts and cancellations in Go - Alex Edwards, accessed on March 10, 2026, https://www.alexedwards.net/blog/how-to-manage-database-timeouts-and-cancellations-in-go
20. Extractors Guide for Middleware - Fiber, accessed on March 10, 2026, https://docs.gofiber.io/blog/fiber-v3-extractors-guide/
21. extractors package - github.com/gofiber/fiber/v3/extractors - Go Packages, accessed on March 10, 2026, https://pkg.go.dev/github.com/gofiber/fiber/v3/extractors
22. Building a Scalable Monolithic Backend with Go Fiber: Folder Structure Explained - Medium, accessed on March 10, 2026, https://medium.com/@christian.asterisk/building-a-scalable-monolithic-backend-with-go-fiber-folder-structure-explained-5d1023eafa5e
23. How to Build REST APIs in Go with Echo - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-07-go-rest-api-echo/view
24. How to implement clean architecture in Golang (EN) | by Ruangyot Nanchiang | Medium, accessed on March 10, 2026, https://medium.com/@rayato159/how-to-implement-clean-architecture-in-golang-en-f50d66378ebf
25. The Repository pattern in Go: a painless way to simplify your service logic - Three Dots Labs, accessed on March 10, 2026, https://threedots.tech/post/repository-pattern-in-go/
26. Database Connections with GORM in Golang: Opening, Closing and Connection Pooling | by Feldy Judah Kambey | Medium, accessed on March 10, 2026, https://medium.com/@feldyjudahk/database-connections-with-gorm-in-golang-opening-closing-and-connection-pooling-277043e1d568
27. Best practice for Database Open and Close connection - Getting Help - Go Forum, accessed on March 10, 2026, https://forum.golangbridge.org/t/best-practice-for-database-open-and-close-connection/19021
28. How to Implement Connection Pooling in Go for PostgreSQL - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-07-go-postgresql-connection-pooling/view
29. 2 Connection Strategies for Database Applications - Oracle, accessed on March 10, 2026, https://docs.oracle.com/en/database/oracle/oracle-database/23/adfns/connection_strategies.html

30. How to Use GORM Effectively in Go - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-07-go-gorm-orm/view
31. Does Gorm automatically close the connection? - Stack Overflow, accessed on March 10, 2026, https://stackoverflow.com/questions/69795462/does-gorm-automatically-close-the-connection
32. How do I handle opening/closing Db connection in a Go app? - Stack Overflow, accessed on March 10, 2026, https://stackoverflow.com/questions/40587008/how-do-i-handle-opening-closing-db-connection-in-a-go-app
33. Managing connections - The Go Programming Language, accessed on March 10, 2026, https://go.dev/doc/database/manage-connections
34. Mastering Database Connection Pooling in Go: Performance Best Practices - Dev.to, accessed on March 10, 2026, https://dev.to/nithinbharathwaj/mastering-database-connection-pooling-in-go-performance-best-practices-4mic
35. Optimizing Database Connections in Go: Improving Throughput by Managing Open Connections Efficiently | by Byron Cabrera | Medium, accessed on March 10, 2026, https://medium.com/@ullauri.byron/optimizing-database-connections-in-go-improving-throughput-by-managing-open-connections-7d90b2eb9169
36. Multi-Tenancy Database Patterns with examples in Go | by Rost Glukhov - Medium, accessed on March 10, 2026, https://medium.com/@rosgluk/multi-tenancy-database-patterns-with-examples-in-go-ade087d642c8
37. Dynamically switching between databases. · go-gorm gorm · Discussion #5564 - GitHub, accessed on March 10, 2026, https://github.com/go-gorm/gorm/discussions/5564
38. Multitenant application with golang and gorm - Reddit, accessed on March 10, 2026, https://www.reddit.com/r/golang/comments/ug1veo/multitenant_application_with_golang_and_gorm/
39. REST API Consumption Patterns - Oracle Help Center, accessed on March 10, 2026, https://docs.oracle.com/en/cloud/paas/integration-cloud/rest-adapter/consume-rest-api.html
40. Introduction | Auth0 Authentication API, accessed on March 10, 2026, https://auth0.com/docs/api/authentication
41. Understanding the Adapter Design Pattern in Go: A Practical Guide - Medium, accessed on March 10, 2026, https://medium.com/design-bootcamp/understanding-the-adapter-design-pattern-in-go-a-practical-guide-a595b256a08b
42. Adapter design pattern in go - DEV Community, accessed on March 10, 2026, https://dev.to/ansu/adapter-design-pattern-in-go-3hbm
43. How to Build a Custom Go Client For a REST API - DEV Community, accessed on

March 10, 2026, https://dev.to/exceptionless/how-to-build-a-custom-go-client-for-a-rest-api-2b1a
44. New Client Deep Dive - Fiber Documentation, accessed on March 10, 2026, https://docs.gofiber.io/blog/fiber-v3-client-deep-dive/
45. New Client Deep Dive - Fiber Documentation, accessed on March 10, 2026, https://docs.gofiber.io/blog/fiber-v3-client-deep-dive
46. Building a Generic Authentication System for API Integrations in Go | by Eugene Nikolaev, accessed on March 10, 2026, https://satorsight.medium.com/building-a-generic-authentication-system-for-api-integrations-in-go-9fa014b6d717
47. How to Parse Dynamic JSON with Unknown Structure in Go - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-23-go-parse-dynamic-json/view
48. Working with Dynamic or Unknown JSON Structures in Go | CodeSignal Learn, accessed on March 10, 2026, https://codesignal.com/learn/courses/handling-json-in-go-1/lessons/working-with-dynamic-or-unknown-json-structures-in-go
49. Golang Reflection: The Guide to Runtime Type Inspection, Manipulation, and Best Practices, accessed on March 10, 2026, https://medium.com/@mojimich2015/golang-reflection-the-guide-to-runtime-type-inspection-manipulation-and-best-practices-303087684576
50. How to Use Reflection for Dynamic JSON Parsing in Go - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-01-25-reflection-dynamic-json-parsing-go/view
51. The Hidden Cost of Reflection in Go — Why Your Code Is Slower Than You Think - DEV Community, accessed on March 10, 2026, https://dev.to/devflex-pro/the-hidden-cost-of-reflection-in-go-why-your-code-is-slower-than-you-think-41ee
52. What is the realistic impact on performance when using 'reflect' : r/golang - Reddit, accessed on March 10, 2026, https://www.reddit.com/r/golang/comments/1i81t5l/what_is_the_realistic_impact_on_performance_when/
53. Handling Large JSON Data Efficiently (Streaming Response) | by Somnathgouda - Medium, accessed on March 10, 2026, https://medium.com/@somnathgouda73/handling-large-json-data-efficiently-streaming-response-853049dbe6f1
54. [Proposal]: Add buffered streaming support · Issue #3127 · gofiber/fiber - GitHub, accessed on March 10, 2026, https://github.com/gofiber/fiber/issues/3127
55. What's New in v3 | Fiber, accessed on March 10, 2026, https://docs.gofiber.io/next/whats_new/
56. Best practice for large JSON response · Issue #809 · gofiber/fiber - GitHub, accessed on March 10, 2026, https://github.com/gofiber/fiber/issues/809
57. Need to stream large files to S3 using Go Fiber/Fasthttp : r/golang - Reddit,

accessed on March 10, 2026, https://www.reddit.com/r/golang/comments/131bq42/need_to_stream_large_files_to_s3_using_go/
58. How to Set Up a Next.js + PostgreSQL + Redis Stack with Docker Compose - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-02-08-how-to-set-up-a-nextjs-postgresql-redis-stack-with-docker-compose/view
59. Part2: Setup production docker and docker-compose with Node.js, MongoDB, Redis, and Next.js | by Zied Hamdi - https://github.com/ziedHamdi | Medium, accessed on March 10, 2026, https://medium.com/@zhamdi/part2-setup-production-docker-and-docker-compose-with-node-js-mongodb-redis-and-next-js-41e93c45d24e
60. BUN your NEXTJS project as a docker container - YouTube, accessed on March 10, 2026, https://www.youtube.com/watch?v=7fPihOaVQF8
61. Docker Compose in Action : A Real Case Study for Nextjs Projects - DEV Community, accessed on March 10, 2026, https://dev.to/brokarim/docker-compose-in-action-a-real-case-study-for-nextjs-projects-3h9p
62. A cleaner way to run `next build` with a database in Docker Compose : r/nextjs - Reddit, accessed on March 10, 2026, https://www.reddit.com/r/nextjs/comments/1m468cl/a_cleaner_way_to_run_next_build_with_a_database/
63. How to Implement Caching Strategies in Go Applications - OneUptime, accessed on March 10, 2026, https://oneuptime.com/blog/post/2026-02-01-go-caching-strategies/view
64. Easily encrypt data at rest in relational databases using drop-in Gorm Serializers. - GitHub, accessed on March 10, 2026, https://github.com/mjpitz/gorm-encryption
65. Anyone doing column-level encryption? : r/golang - Reddit, accessed on March 10, 2026, https://www.reddit.com/r/golang/comments/v6g4dw/anyone_doing_columnlevel_encryption/